

Лабораторна робота №5

Розширити попередню лабораторну роботу додавши сховище для зберігання даних у вигляді озерного сховища даних.

Ключові компоненти для побудови озерного сховища даних:

Apache Iceberg — це популярний відкритий формат таблиць, який надає вашому озеру даних сучасні функції баз даних: ACID транзакції, еволюцію схем, еволюцію розділів, функцію «подорожі в часі», тегування та розгалуження.

Apache Polaris — це REST-каталог Iceberg. Його можна розглядати як центральний «мозок», що відстежує рівень метаданих Iceberg.

Trino — це швидкий розподілений механізм SQL-запитів. Він може записувати та зчитувати дані з таблиць Iceberg завдяки своєму коннектору Iceberg.

MinIO забезпечує сховище, сумісне з S3, без необхідності використання AWS

Ось типовий алгоритм взаємодії:

1. Trino звертається до Polaris, коли потрібно знайти таблиці та їхні найновіші метадані.
2. Polaris зберігає всі метадані Iceberg у MinIO і також надає їх звітти.
3. Trino записує та зчитує дані Iceberg, використовуючи безпосередньо MinIO (для підвищення продуктивності).

Етапи виконання:

До файлу *docker-compose.yml* з попередньої лабораторної роботи додайте наступні сервіси:

```
services:

  polaris:
    image: apache/polaris:latest
    platform: linux/amd64
    ports:
      - "8181:8181"
      - "8182:8182"
    networks:
      - local-iceberg-lakehouse
    environment:
      AWS_ACCESS_KEY_ID: admin
      AWS_SECRET_ACCESS_KEY: password
      AWS_REGION: dummy-region
```

```

AWS_ENDPOINT_URL_S3: http://minio:9000
AWS_ENDPOINT_URL_STS: http://minio:9000
POLARIS_BOOTSTRAP_CREDENTIALS: default-realm,root,secret
polaris.features.DROP_WITH_PURGE_ENABLED: true # allow dropping tables from the
SQL client
  polaris.realm-context.realms: default-realm
healthcheck:
  test: ["CMD", "curl", "http://localhost:8181/healthcheck"]
  interval: 5s
  timeout: 10s
  retries: 5

trino:
  image: trinodb/trino:latest
  ports:
    - "8080:8080"
  environment:
    - TRINO_JVM_OPTS=-Xmx2G
  networks:
    - local-iceberg-lakehouse
  volumes:
    - ./trino/catalog:/etc/trino/catalog

minio:
  image: minio/minio:latest
  volumes:
    - ./minio_data:/data
  environment:
    AWS_ACCESS_KEY_ID: admin
    AWS_SECRET_ACCESS_KEY: password
    AWS_REGION: dummy-region
    MINIO_ROOT_USER: admin
    MINIO_ROOT_PASSWORD: password
    MINIO_DOMAIN: minio
  networks:
    local-iceberg-lakehouse:
      aliases:
        - warehouse.minio
  ports:
    - "9001:9001"
    - "9000:9000"
  command: ["server", "/data", "--console-address", ":9001"]

minio-client:
  image: minio/mc:latest
  depends_on:
    - minio
  networks:
    - local-iceberg-lakehouse
  volumes:
    - /tmp:/tmp

```

```

environment:
  AWS_ACCESS_KEY_ID: admin
  AWS_SECRET_ACCESS_KEY: password
  AWS_REGION: dummy-region
entrypoint: >
  /bin/sh -c "
    until (mc alias set minio http://minio:9000 admin password) do echo
'...waiting...' && sleep 1; done;
    mc rm -r --force minio/warehouse;
    mc mb minio/warehouse;
    mc anonymous set public minio/warehouse;
    tail -f /dev/null
  "

networks:
  local-iceberg-lakehouse:
    name: local-iceberg-lakehouse

```

При потребі скорегуйте мережу контейнерів.

Перш ніж Trino зможе під'єднатися до Polaris, його потрібно налаштувати. Створіть підкаталог для файлу конфігурації:

```
mkdir -p ./trino/catalog/
```

Створіть файл *iceberg.properties* з наступними налаштуваннями:

```

connector.name=iceberg
iceberg.catalog.type=rest
iceberg.rest-catalog.uri=http://polaris:8181/api/catalog/
iceberg.rest-catalog.warehouse=polariscatalog
iceberg.rest-catalog.vended-credentials-enabled=true
iceberg.rest-catalog.security=OAUTH2
iceberg.rest-catalog.oauth2.credential=root:secret
iceberg.rest-catalog.oauth2.scope=PRINCIPAL_ROLE:ALL

# required for Trino to read from/write to S3
fs.native-s3.enabled=true
s3.endpoint=http://minio:9000
s3.region=dummy-region

```

Запустіть стек наступною командою:

```
docker compose up
```

На вашому комп'ютері запрацюють такі служби:

- Trino Web UI: <http://localhost:8080>
- MinIO UI: <http://localhost:9001> (admin/password)
- MinIO API: <http://localhost:9001>
- Polaris: <http://localhost:9001>

Polaris потребує початкового налаштування:

Створити каталог Iceberg

Потрібно створити каталог Iceberg у Polaris, але для цього спочатку необхідно отримати токен доступу. У новому терміналі виконайте таку команду:

```
ACCESS_TOKEN=$(curl -X POST \
  http://localhost:8181/api/catalog/v1/oauth/tokens \
  -d
'grant_type=client_credentials&client_id=root&client_secret=secret&scope=PRINCIPAL_ROLE:ALL' \
  | jq -r '.access_token')
```

Під час створення каталогу потрібно вказати Polaris, де зберігати дані та як отримати до них доступ. У нашому випадку все буде зберігатися в MinIO.

```
curl -i -X POST \
  -H "Authorization: Bearer $ACCESS_TOKEN" \
  http://localhost:8181/api/management/v1/catalogs \
  --json '{
    "name": "polariscatalog",
    "type": "INTERNAL",
    "properties": {
      "default-base-location": "s3://warehouse",
      "s3.endpoint": "http://minio:9000",
      "s3.path-style-access": "true",
      "s3.access-key-id": "admin",
      "s3.secret-access-key": "password",
      "s3.region": "dummy-region"
    },
    "storageConfigInfo": {
      "roleArn": "arn:aws:iam::000000000000:role/minio-polaris-role",
      "storageType": "S3",
      "allowedLocations": [
        "s3://warehouse/*"
      ]
    }
  }'
```

Перевірте, чи каталог було правильно створено в Polaris:

```
curl -X GET http://localhost:8181/api/management/v1/catalogs \
-H "Authorization: Bearer $ACCESS_TOKEN" | jq
```

Результат повинен виглядати так:

```
{
  "catalogs": [
    {
      "type": "INTERNAL",
      "name": "polariscatalog",
      "properties": {
        "s3.path-style-access": "true",
        "s3.access-key-id": "admin",
        "s3.secret-access-key": "password",
        "default-base-location": "s3://warehouse",
        "s3.region": "dummy-region",
        "s3.endpoint": "http://minio:9000"
      },
      "createTimestamp": 1750257800389,
      "lastUpdateTimestamp": 1750257800389,
      "entityVersion": 1,
      "storageConfigInfo": {
        "roleArn": "arn:aws:iam::000000000000:role/minio-polaris-role",
        "externalId": null,
        "userArn": null,
        "region": null,
        "storageType": "S3",
        "allowedLocations": [
          "s3://warehouse/*",
          "s3://warehouse"
        ]
      }
    }
  ]
}
```

Налаштування прав доступу

Polaris має досить просунуту модель контролю доступу на основі ролей.

Якщо термін дії вашого токена доступу закінчився, ви можете створити новий за допомогою команди, наведеної вище.

```
# Create a catalog admin role
curl -X PUT http://localhost:8181/api/management/v1/catalogs/polariscatalog/catalog-
roles/catalog_admin/grants \
-H "Authorization: Bearer $ACCESS_TOKEN" \
--json '{"grant":{"type":"catalog", "privilege":"CATALOG_MANAGE_CONTENT"}}'
```

```
# Create a data engineer role
curl -X POST http://localhost:8181/api/management/v1/principal-roles \
-H "Authorization: Bearer $ACCESS_TOKEN" \
--json '{"principalRole":{"name":"data_engineer"}}'

# Connect the roles
curl -X PUT http://localhost:8181/api/management/v1/principal-roles/data_engineer/catalog-roles/polariscatalog \
-H "Authorization: Bearer $ACCESS_TOKEN" \
--json '{"catalogRole":{"name":"catalog_admin"}}'

# Give root the data engineer role
curl -X PUT http://localhost:8181/api/management/v1/principals/root/principal-roles \
-H "Authorization: Bearer $ACCESS_TOKEN" \
--json '{"principalRole": {"name":"data_engineer"}}'
```

Перевірте, чи роль була правильно призначена суб'єкту root:

```
url -X GET http://localhost:8181/api/management/v1/principals/root/principal-roles -H
"Authorization: Bearer $ACCESS_TOKEN" | jq
```

Результат повинен бути таким:

```
{
  "roles": [
    {
      "name": "service_admin",
      "federated": false,
      "properties": {},
      "createTimestamp": 1751733238263,
      "lastUpdateTimestamp": 1751733238263,
      "entityVersion": 1
    },
    {
      "name": "data_engineer",
      "federated": false,
      "properties": {},
      "createTimestamp": 1751733315678,
      "lastUpdateTimestamp": 1751733315678,
      "entityVersion": 1
    }
  ]
}
```

Як користуватися цією програмою

Настав час створити таблицю Iceberg і виконати кілька запитів. Відкрийте сеанс у Trino,

який під'єднається до каталогу Polaris Iceberg.

```
docker compose exec -it trino trino --server localhost:8080 --catalog iceberg
```

У командному рядку Trino створіть схему (яка відповідає простору імен у Polaris), а потім активуйте її.

```
-- Create a schema first (a namespace in Polaris).
CREATE SCHEMA db;

-- Activate the schema
USE db;
```

Далі створіть таблицю, скорегуйте схему таблиці відповідно до вашої структури даних:

```
CREATE TABLE customers (
  customer_id BIGINT,
  first_name VARCHAR,
  last_name VARCHAR,
  email VARCHAR
);
```

Давайте введемо кілька записів у цю таблицю:

```
INSERT INTO customers (customer_id, first_name, last_name, email)
VALUES (1, 'Rey', 'Skywalker', 'rey@rebelscum.org'),
       (2, 'Hermione', 'Granger', 'hermione@hogwarts.edu'),
       (3, 'Tony', 'Stark', 'tony@starkindustries.com');
```

Коли ми зробимо запит до таблиці, з'являться записи.

```
SELECT * FROM customers;
```

Корисні посилання:

- [Оригінал гайду англійською мовою](#)
- [Apache Iceberg](#)
- [Apache Polaris](#)
- [Trino](#)
- [MinIO](#)